

# fox2db — Regler-Modell (Sofar / Waveshare ESP32-S3)

---

Lastfolge-Überschussregler für eine Photovoltaik-Anlage mit zwei Batteriespeichern. Der Regler schaltet eine quantisierte elektrische Last (EBox-Heizstäbe) so, dass der Netzaustausch nahe Null bleibt ("Sweet-Spot"), ohne die autarke Sofar-Batterie zu stören.

## Klassifikation

Zeitdiskreter, nichtlinearer hybrider Automat (Mealy-Maschine mit Hysterese) — KEIN PID und KEIN LTI-System. Der Aktuator ist diskret (8 Leistungsstufen über 3 Relais-Bits), daher Totband + Ratenbegrenzung statt kontinuierlicher Stellgröße. Takt: 60 s.

## Ein- und Ausgänge

Eingänge : pcc    Netzaustausch            [W]    (+ = Einspeisung)  
          ebox    gemessene EBox-Last    [W]  
          bat1    Sofar-Batterieleistung[W]    (autark, nur gelesen)  
                  (SOC1 wird NICHT verwendet; SOC2 steuert separate Guards)  
Zustand : s       Relais-State 0..7  
          c       Stabilisierungs-Zähler  
          E\_-1    Überschuss des Vortakts (für Trend)  
Ausgang : s       ein State 0..7 -> EBox-Leistung P[s]

## Anlagen-Grenzen

Sicherung 3x50A @230V .... 34500 W Netzanschluss  
PV-Spitzenleistung ..... 35000 W  
EBox-Max (State 7) ..... 11400 W (= 33% Sicherung, 33% PV)  
Bat1 (Sofar) ..... 5 kWh  
Bat2 (EBox) ..... 30 kWh (@State7: +0.63 %/min)

# Mathematisches Modell

---

## Leistungs-Map und Rang

$$P = (0, 3000, 3650, 6650, 3900, 7100, 7800, 11400) \text{ W}$$

$$\Pi \text{ (aufsteigend)} = (0, 3000, 3650, 3900, 6650, 7100, 7800, 11400)$$

## Schritt-Rekursion $x_k=(s,c,E)$

$$e_{\text{eff}} = \max(e_k, P_{s_{k-1}}) \quad (s_{k-1} > 0)$$

$$E_k = p_k + e_{\text{eff}} + b_k$$

$$\tau_k = 0 \quad (E_k < E_{\min}), \quad \tau_k = Q(E_k + G) \quad \text{sonst}$$

$$Q(B) = \text{groesster State } s \text{ mit } P_s \leq B$$

## Hochrampe (max. eine Leistungsstufe / Takt)

$$\tilde{\tau}_k = \min(\tau_k, \hat{s}(\text{rang}(s_{k-1}) + 1))$$

## Schalt-/Schutzlogik (Reihenfolge = Priorität)

|                 |   |                                  |                    |
|-----------------|---|----------------------------------|--------------------|
| EMERGENCY_FORCE | : | pcc < -1020 und runter           | -> sofort schalten |
| SWEET_SPOT_HOLD | : | hoch &  pcc  < 160 & bat1 > -310 | -> halten          |
| TREND_BLOCK     | : | hoch & dE/dt < -20 W/s           | -> halten          |
| BAT_GUARD_BLOCK | : | hoch & bat1 < -220 W             | -> halten          |
| STABILIZING     | : | runter & c < 2 Takte             | -> halten          |
| HYSTERESIS      | : | runter &  dP  < 505 W            | -> halten          |

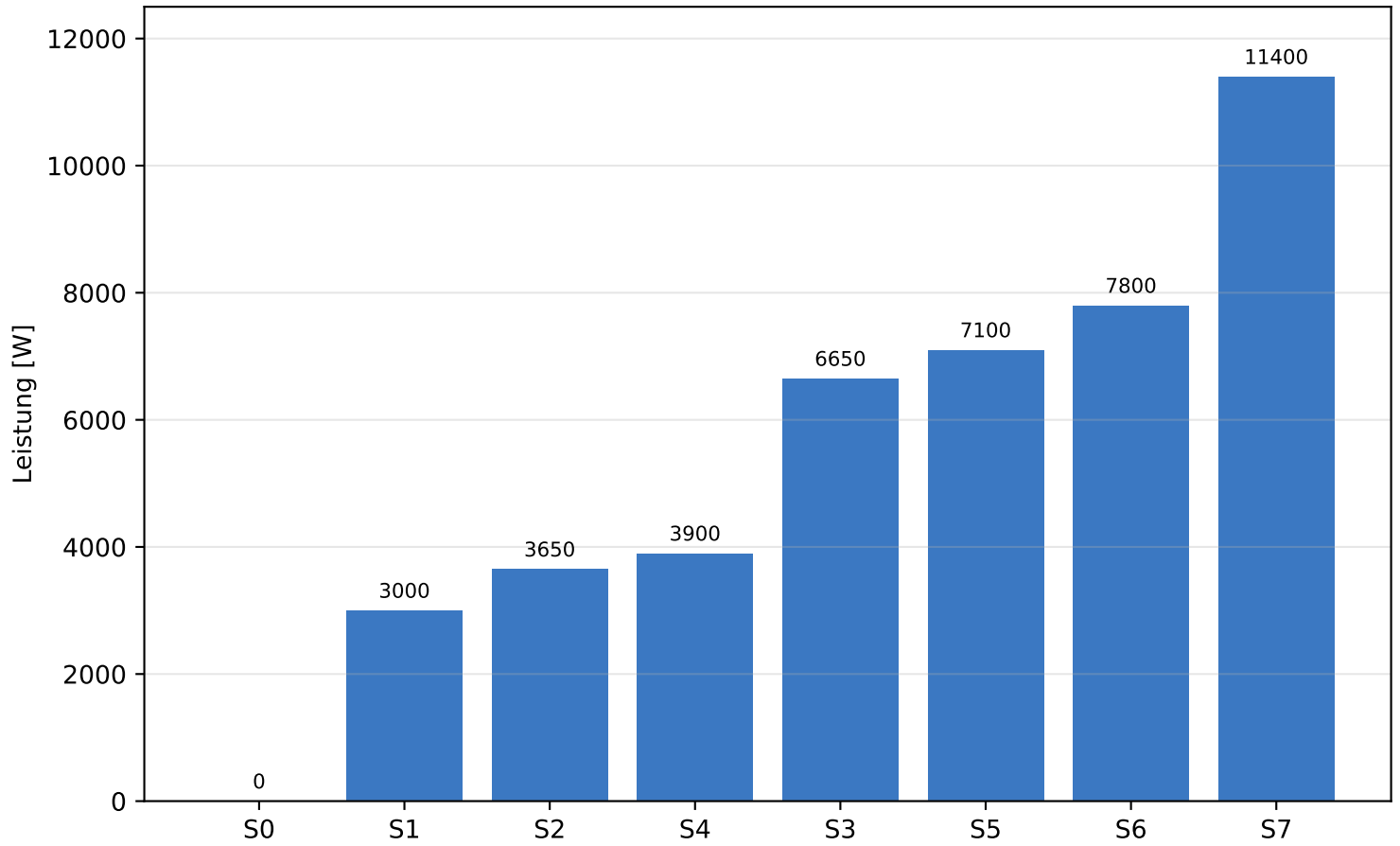
## Konstanten

$$E_{\min} = 1200, G = 1200, H = 505, N = 2 \text{ [W]}$$

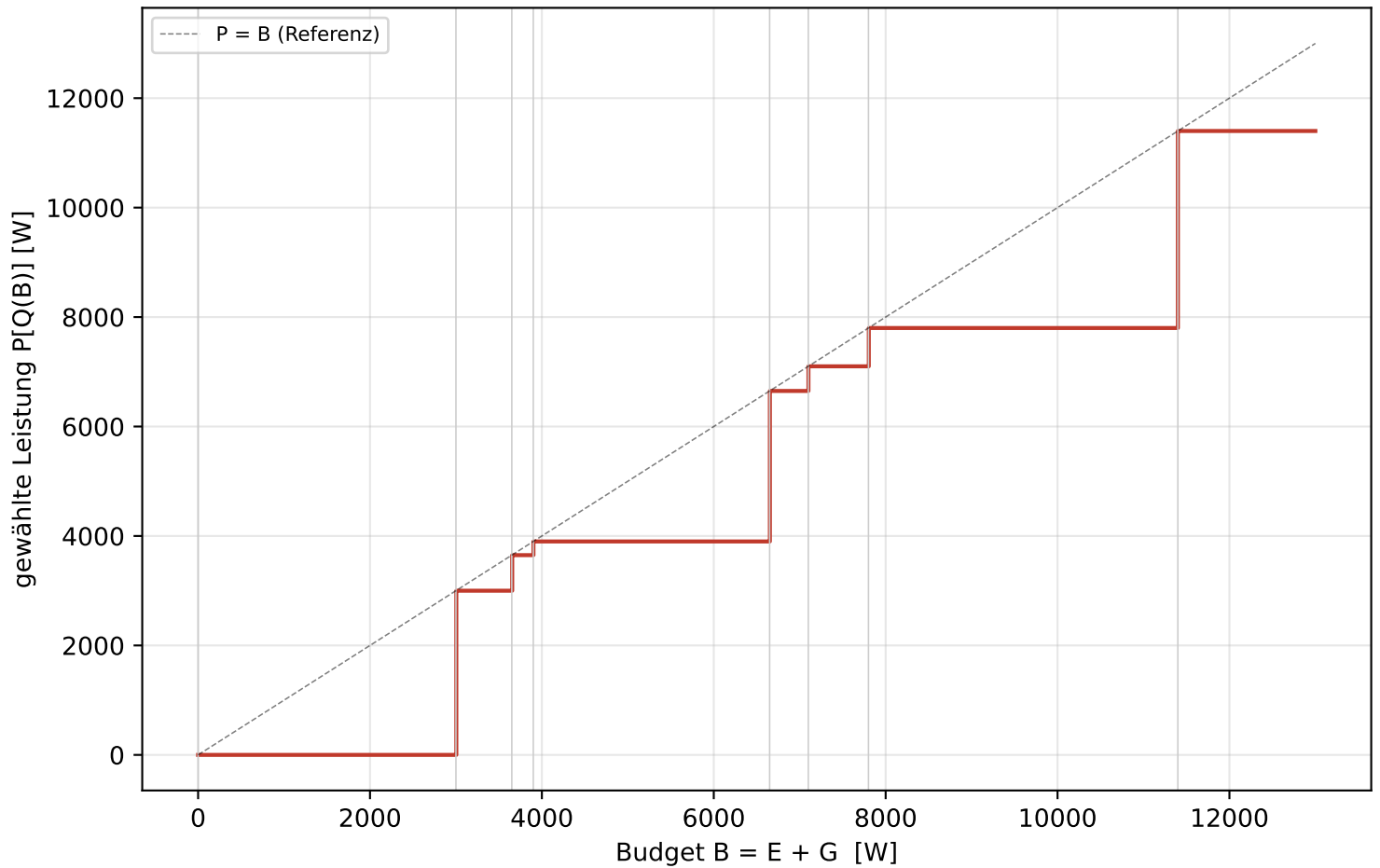
## Totband (Gleichgewicht, Last an $e_{\text{eff}}=P_s$ )

$$-G < p < (\Pi^+(s) - P_s) - G$$

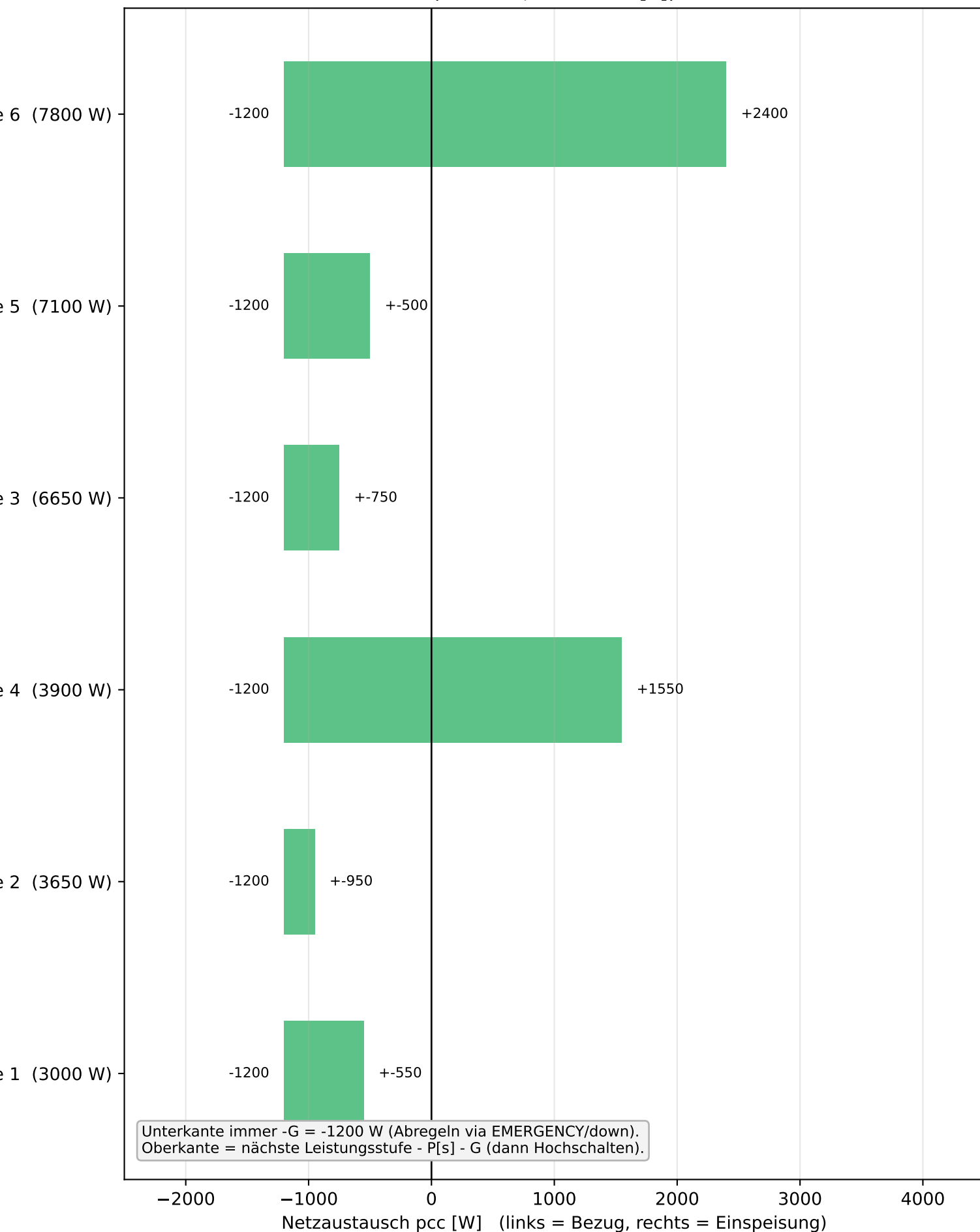
Leistungs-Map P[s] (nach Leistung sortiert)



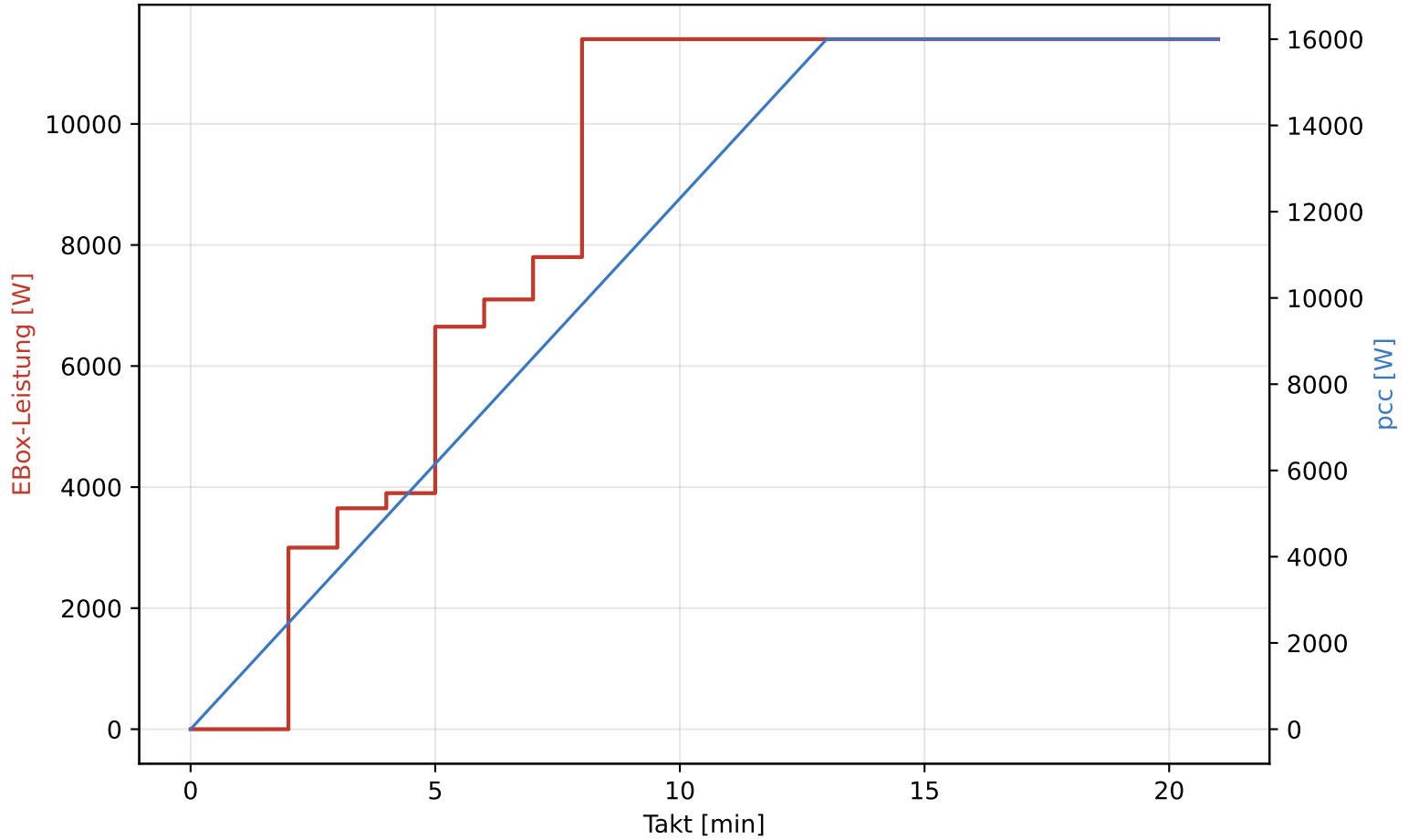
Quantisierer Q(B): höchster State mit  $P[s] \leq B$



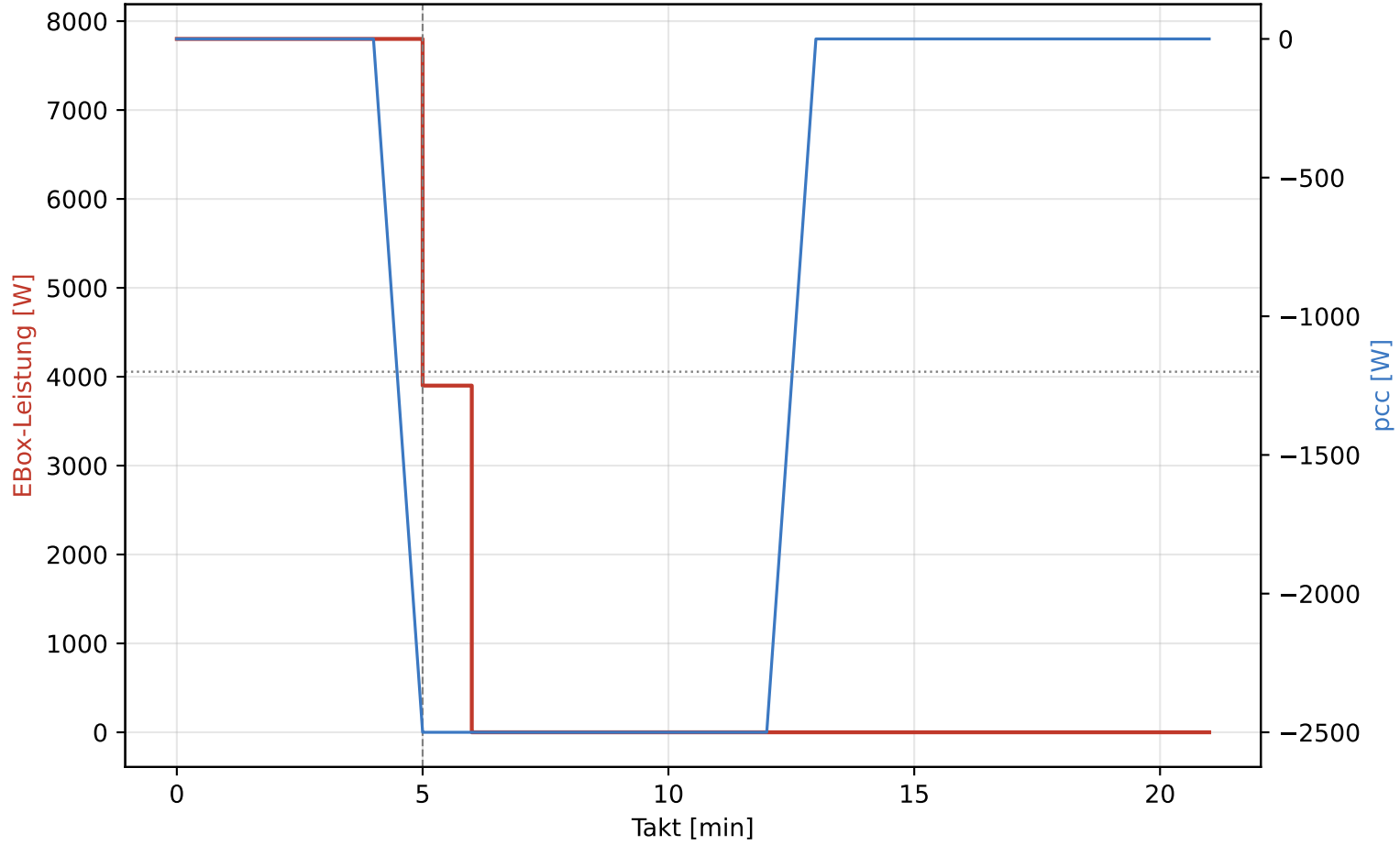
Totband (Sweet-Spot): pcc-Bereich, in dem ein State gehalten wird  
(Last an, ebox = P[s])



(a) Hochlauf: Rampenbegrenzung — max. eine Leistungsstufe pro Takt



(b) Lastsprung im Sweet-Spot: -2500 W ab Takt 5 (Wärmepumpe)



# Validierung gegen Realdaten

---

Die Referenz `step_full_literal` wurde one-step-ahead gegen die echten Entscheidungen des laufenden ESP (Tabelle `pv_decision_log`, `version='wvshare'`) geprüft: pro Takt wird (`state_from`, `pcc`, `bat1`, `ebox`) eingespeist und der vorhergesagte Folge-State verglichen.

## Ergebnis (Messfenster 2026-06-10, Mittag)

STATE-Treffer (one-step) : 55 / 55 (100.0 %)  
excess-Formel exakt : 54 / 55 ( 98.2 %)  
Grund + Modifizier exakt : 54 / 55 ( 98.2 %)  
abgedeckte Entscheidungen: `POWER_MATCHING`, `INSUFFICIENT_EXCESS`,  
`RAMP_LIMITED`, `SWEET_SPOT_HOLD`, `TREND_BLOCK`, `BAT_GUARD_BLOCK`,  
`EMERGENCY_FORCE`, `STABILIZING`

Die eine Abweichung (12:25) ist ein Quantisierer-Randfall: eine  $\sim 2$  W Rundungsdifferenz im Überschuss verschob das Ziel um eine Stufe, die sofort durch `STABILIZING` zurückgehalten wurde -> identischer End-State.

## Hinweis zur Implementierung

Der C-Code (`fox2db_logic.h`) vergleicht bei der Hochrampe über die State-NUMMER statt den Leistungs-RANG. Da  $P$  nicht monoton in der Nummer ist ( $\text{State}_4=3900\text{W} < \text{State}_3=6650\text{W}$ ), weicht das in Randfällen vom rang-monotonen Modell ab. `step_full_literal` bildet den C-Code 1:1 ab; `step_full_rank` ist die saubere Form.

## Artefakte

|                                |                                           |
|--------------------------------|-------------------------------------------|
| <code>fox2db_logic.h</code>    | ESP-Steuerlogik (C++)                     |
| <code>fox2dbEasy.py</code>     | 1:1 Python-Port (Schatten)                |
| <code>fox2db_model.py</code>   | math. Referenz + Grenzen + 49 Selbsttests |
| <code>validate_model.py</code> | DB-Validierung (one-step-ahead)           |